

Redis原理—1.Redis数据结构

Original 东阳马生 东阳马生架构 2024年12月8日 21:28 广东

大纲(9036字)

1.Redis的数据结构

2.Redis的SDS

3.Redis的链表

4.Redis的字典

5.Redis的跳跃表

6.Redis的整数集合

7.Redis的压缩列表

8.Redis的对象

9.Redis对象的几个关键属性

10.Redis的单线程为什么这么快

11.Redis的典型应用场景和说明

12.Redis的相关命令说明

1.Redis的数据结构

数据结构有：**简单动态字符串SDS**、**链表**、**字典**、**跳跃表**、**整数集合**、**压缩列表**。

2.Redis的SDS

(1)SDS的应用

(2)SDS的结构

(3)SDS的优点

(4)什么是空间预分配

(5)什么是惰性空间释放

(6)SDS是二进制安全的

(1)SDS的应用

SDS除了用来保存Redis的字符串值外，**AOF缓冲区**、**客户端状态中的输入缓冲区**都是由SDS实现的。

(2)SDS的结构

```
1 //sdshdr的结构
2 int len;//SDS保存字符串的长度，占4个字节
3 int alloc;//数组中未使用的字节数，占4个字节
4 char buf[];//保存字符串的字节数组 + 分割符"\0"占一个字节
```

(3)SDS的优点

一.**常数复杂度O(1)获取字符串长度**，C字符串获取长度为O(n)，SDS只需获取len属性即可

二.**杜绝缓冲区溢出**，拼接字符串之前，先检查alloc属性，不够则扩展SDS空间

三.通过空间预分配和惰性空间释放，减少修改字符串带来的内存重分配次数

四.二进制安全

五.兼容部分C字符串函数

(4)什么是空间预分配

空间预分配用于优化SDS增长操作，具体来说就是扩展SDS时，分配额外的未使用空间。

如果SDS长度小于1MB，则分配和len属性同样大小的未使用空间，即buf数组长变为： $2len + 1$ 。如果SDS长度大于1MB，则分配1MB的未使用空间。

通过空间预分配，在扩展SDS空间之前，如果未使用空间足够，则无需执行内存空间重分配。这样SDS就可以将连续增长N次字符串所需的内存重分配次数从N次降为最多N次。

(5)什么是惰性空间释放

缩短SDS时，不立即使用内存重分配来回收多出的字节，而是使用alloc属性记录起来将来使用。

(6)SDS是二进制安全的

使用二进制安全的SDS，使得Redis不仅可以保存文本数据，还可以保存任意格式的二进制数据。

3.Redis的链表

(1)链表的应用

(2)链表的结构

(3)Redis链表的特性

(1)链表的应用

发布与订阅、慢查询、监视器、保存多个客户端状态(redisClient)、构建客户端输出缓冲区、列表键

(2)链表的结构

```
1 //ListNode的结构如下:
2 listNode *prev; //前置结点
3 listNode *next; //后置结点
4 void *value; //结点值
5
6 //List的结构如下:
7 listNode *head; //表头结点
8 listNode *tail; //表尾结点
9 long len; //链表包含结点数量
```

每个链表结点由一个listNode结构表示。每个结点都有一个指向前置结点和后置结点的指针，所以Redis的链表是双端链表。每个链表使用一个list结构表示，这个结构带有表头指针、表尾指针以及链表长度等信息。因为链表表头结点的前置结点和表尾结点的后置结点都指向NULL，所以是无环链表。

注意：Redis的链表可以保存各种不同的值。

(3)Redis链表的特性

- 一.双端：获取结点的前后结点
- 二.无环：对链表的访问以NULL为终点
- 三.带表头指针和表尾指针，获取链表表头、表尾结点的时间复杂度为O(1)
- 四.带链表长度计数器
- 五.多态：通过void *指针保存结点，可以保存各种不同类型的值

4.Redis的字典

(1)字典的应用

(2)字典的结构

(3)哈希的算法

(4)哈希冲突的解决

(5)rehash的步骤

(6)哈希表的负载因子

(7)渐进式rehash

(1)字典的应用

Redis的数据库就是使用字典作为底层实现的，Redis的哈希键也使用了字典作为底层实现，其中Redis的字典是使用哈希表作为底层实现的。

(2)字典的结构

```
1 //dict字典的结构如下:
2 dict_t ht[2]; //哈希表数组
3 int rehashidx; //rehash索引
4
5 //dict_t哈希表的结构如下:
6 dictEntry **table; //哈希结点数组
7 long size; //哈希表大小
8 long used; //哈希结点数量
9 long sizemask; //哈希表大小掩码, 用于计算索引值
10
11 //dictEntry哈希结点的结构如下:
12 void *key; //键, 8字节
13 union v; //值, 8字节
14 dictEntry *next; //下一个结点, 8字节
```

一.ht属性通常使用ht[0]，rehash时使用ht[1]

二.rehashidx属性用来实现渐进式rehash

三.next属性是指向另一个哈希表结点的指针，这个指针可以将多个哈希值相同的键值对连接在一起，以此来解决哈希冲突的问题

(3)哈希的算法

将一个新的键值对添加到字典时，先根据键值对的键算出哈希值和索引值。然后根据索引值，将包含新键值对的哈希表结点，放到哈希表数组指定的索引上。

Redis使用MurmurHash2算法来计算键的哈希值，该算法速度快，而且有很好的随机分布性。

(4)哈希冲突的解决

Redis的哈希表使用链地址法来解决键冲突。每个哈希表结点都有一个next指针，多个哈希表结点可以用next指针构成一个单向链表。被分配到同一个索引上的多个结点，可以用单向链表连接起来，并且会使用头插法将新结点加到链表中，以更快完成插入。

(5)rehash的步骤

一.为字典的ht[1]哈希表分配空间

如果是扩展操作，那么ht[1]大小等于ht[0].used * 2ⁿ，如果是收缩操作，那么ht[1]大小等于ht[0].used * 2ⁿ。

二.将保存在ht[0]中的所有键值对rehash到ht[1]上

三.ht[0]所有键值对迁移到ht[1]后释放ht[0]

然后将ht[1]设为ht[0]，在ht[1]新建空哈希表。

(6)哈希表的负载因子

哈希表的负载因子 = 哈希表已保存结点数量 / 哈希表大小

在如下情况，程序会自动对哈希表进行扩展操作：

一.当没有执行bgsave或bgrewriteaof时，负载因子大于等于1

二.当正在执行bgsave或bgrewriteaof时，负载因子大于等于5

因为在bgsave或bgrewriteaof过程中，Redis需要创建当前服务进程的子进程，而大多数操作系统会采用写时复制技术来优化子进程的使用效率。

所以，在子进程存在期间，服务器会提高执行扩展操作的负载因子，从而尽可能避免在子进程存在期间进行哈希表扩展操作，避免不必要的内存写入，最大程度上节约内存。

此外，当哈希表的负载因子小于0.1时，程序会自动开始对哈希表进行收缩操作。

(7)渐进式rehash

为避免rehash对服务器性能造成影响，服务器不是一次性将ht[0]里所有的键值对全部rehash到ht[1]上，而是分多次、渐进式、分而治之地将ht[0]里面的键值对慢慢rehash到ht[1]。

将rehash键值对所需的工作均摊到对字典的每个添加、删除、查找和更新操作上，从而避免集中式rehash而带来的庞大计算量。

字典中维持的索引计数器变量rehashidx，记录了正在rehash到的索引。在rehash被触发后，即使没有收到新请求，Redis也会有定时任务触发rehash操作，而且每次不超过1ms。

在渐进式rehash进行期间，字典会同时使用ht[0]和ht[1]两个哈希表。字典的删除、查找、更新操作会在两个哈希表上进行，而新增操作会在ht[1]进行。

5.Redis的跳跃表

(1)跳跃表的实现

(2)跳跃表的应用

(3)跳跃表的结构

(4)跳跃表的图示

(5)跳跃表的说明

(1)跳跃表的实现

跳跃表通过在每个结点中维持多个指向其他结点的指针，达到快速访问结点的目的。

(2)跳跃表的应用

Redis只有两个地方用到跳跃表，一个是实现有序集合键，另一个是集群节点中根据槽批量获取键。

(3)跳跃表的结构

```
1 //zskiplist跳跃表的结构如下：
2 header：指向跳表的表头结点
3 tail：指向跳表的表尾结点
4 level：层数最大的结点的层数(表头结点不算)
5 length：跳表长度，即结点数量
6
7 //zskiplistNode跳跃表结点的结构如下：
8 *backward：后退指针
9 score：分值
10 obj：成员对象
11 zskiplistLevel：层数组
12
13 //zskiplistLevel的结构如下：
14 *forward：前进指针
15 span：跨度
```

跳表插入、删除、查找的平均时间复杂度为 $O(\log N)$ ，最坏的时间复杂度为 $O(N)$ 。

压缩列表插入、删除、查找的平均时间复杂度为 $O(N)$ ，最坏的时间复杂度为 $O(N^2)$ 。

整数数组插入、删除的平均时间复杂度为 $O(N)$ ，查找的平均时间复杂度为 $O(\log N)$ 。

(4)跳跃表的图示

每次创建一个新跳跃表结点的时候，程序都会根据幂次定律随机生成一个1~32的值作为level数组大小，这个大小就是层的高度(由L1递增)。

下图中的虚线表示了遍历跳跃表结点的路径，根据跨度为1来选择层中数组来决定下个结点。

(5)跳跃表的说明

遍历操作只用前进指针就可以了。比如从跳跃表的表头结点的第一层出发L1，表头L1会指向第一个结点的指针，到第一个结点的L1会指向第二个结点的指针，定位到第二个结点后，从其L1又可以获取第三个结点的指针，依此类推。

跨度的作用是用来**计算排位**的。在查找某个节点的过程中，将沿途访问过的所有层的跨度累计起来，得到的结果就是目标结点在跳跃表中的排位。

节点的后退指针用于从表尾开始向表头方向访问结点。**跳跃表中所有结点都按分值从小到大排序**，多个结点包含相同分值，**每个跳跃表节点的层高都是1~32之间的随机数**。

6.Redis的整数集合

(1)整数集合的实现

(2)整数集合的结构

(3)整数集合的升级操作

(1)整数集合的实现

整数集合的底层实现为数组，这个数组以有序、无重复方式保存集合元素。在有需要的时候，程序会根据新添加元素类型，改变这个数组的类型。

(2)整数集合的结构

```
1 //intset的结构如下:
2 encoding; //编码
3 length; //数量
4 contents[]; //元素
```

(3)整数集合的升级操作

整数集合的升级操作为整数集合带来了操作上的灵活性，并且尽可能节约内存。

C语言中通常不会将不同类型的值放在同一数据结构里，灵活性是指我们可以将int16_t、int32_t、int64_t类型的整数随意添加到整数集合中。

整数集合能同时保存int16_t、int32_t、int64_t这三种不同的值，且有需要时才升级，这样就可以尽可能节约内存。

整数集合只支持升级操作，不支持降级操作。

7.Redis的压缩列表

(1)压缩列表的实现

(2)压缩列表的结构

(3)压缩列表产生连锁更新的原因

(4)压缩列表总结

(1)压缩列表的实现

压缩列表是Redis为了节约内存而开发的，由一系列特殊编码的连续内存块组成的顺序型数据结构。一个压缩列表可以包含任意多个节点，每个节点可以保存一个字节数组或者一个整数值。

(2)压缩列表的结构

```
1 //压缩列表的结构如下:
2 zlbytes: 整个压缩列表内存字节数, 4字节, 可以理解为列表长度
3 zltail: 压缩列表尾结点距离起始地址有多少字节, 4字节, 可以理解为列表尾偏移量
4 zlen: 压缩列表结点数量, 2字节, 可以理解为列表结点个数
5 entryx: 压缩列表的结点
6 zlend: 标记压缩列表末端, 1字节
7
8 //压缩列表结点的结构如下:
9 previous_entry_length: 前一个节点长度, 实现表尾到表头的遍历, 占1或5个字节
```

- 10 encoding: 记录数据类型和长度, 占1字节
- 11 content: 保存结点的值

(3)压缩列表产生连锁更新的原因

若前一节点的长度小于254字节, 那么previous_entry_length属性占一个字节长。若前一节点的长度大于等于254字节, 那么previous_entry_length属性占5个字节长。所以添加新节点或者删除节点可能会引起连锁更新, 也就是引发多个结点更新。

连锁更新最坏的情况要对压缩列表进行N次空间重分配操作, 而每次空间重分配最坏的时间复杂度为O(N), 所以连锁更新最坏的时间复杂度为O(N^2)。

实际上, 尽管连锁更新复杂度高, 但真正造成性能问题的几率很低, 原因如下:

- 一.压缩列表恰好有多个连续、且长度介于250~253字节的结点, 连锁更新才可能被引发
- 二.即便出现连锁更新, 只要被更新的结点数量不多, 也不影响性能

(4)压缩列表总结

- 一.它是一种为节约内存而开发的顺序型数据结构
- 二.它被用作列表键和哈希键的底层实现
- 三.它可以包含多个结点, 每个结点可以保存一个字节数组或整数值
- 四.添加新结点或删除结点, 可能会引发连锁更新操作, 但几率不高

问题: 整数数组和压缩列表在查找时间复杂度并没有很大优势, 为什么Redis还用?

答: 这体现了Redis"又快又省"中的省, 即节省内存空间。两者都是在内存中分配一块地址连续的空间, 然后把元素一个个紧凑地放在一起。因为元素是挨个连续放置, 所以我们不用通过额外的指针就能把元素串起来, 避免了额外指针带来的空间开销。Redis之所以采用不同的数据结构, 是为了在性能和内存使用效率之间进行平衡。

8.Redis的对象

(1)Redis的对象类型

(2)Redis对象的回收和共享

(3)Redis对象的空转时长

(4)Redis对象的结构

(5)字符串对象的编码

(6)列表对象的编码

(7)哈希对象的编码

(8)集合对象的编码

(9)有序集合的编码

(1)Redis的对象类型

Redis有5种类型的对象: 字符串对象、列表对象、哈希对象、集合对象、有序集合对象。Redis针对不同的使用场景为对象设置多种不同的数据结构实现, 从而优化对象在不同场景下的使用效率。

(2)Redis对象的回收和共享

Redis的对象系统实现了**基于引用计数技术的内存回收机制**。当程序不再使用某个对象的时候，这个对象所占用的内存就会被自动释放。

Redis还通过**引用计数技术**实现了**对象共享机制**。这一机制可以在适当的条件下，通过让多个数据库键共享同一对象来节约内存。

(3)Redis对象的空转时长

Redis的对象带有**访问时间记录信息**，该信息可以用于**计算数据库键的空转时长**。在服务器启用了maxmemory功能下，**空转时长较大的那些键可能会优先被删除**。

(4)Redis对象的结构

Redis中每个对象都由一个redisObject结构表示。这个redisObject结构包括：**type属性、encoding属性、ptr属性、refcount引用计数、lru空转时长**。这个redisObject结构的大小是：**8字节的元数据 + 8字节的指针**。

Redis数据库保存的键值对：**键总是一个字符串对象**。值可以是**字符串对象、列表对象、哈希对象、集合对象、有序集合对象**。

```
1 //type属性的值如下:
2 redis_string
3 redis_list
4 redis_hash
5 redis_set
6 redis_zset
7
8 //encoding属性的值
9 int、embstr、raw => SDS
10 ht                => 字典
11 linkedlist        => 双端链表
12 ziplist            => 压缩列表
13 intset             => 整数集合
14 skiplist           => 跳表和字典
```

通过encoding属性来设定对象所使用的编码，而不是为特定类型的对象关联一种特定编码，极大地提升了Redis的灵活性和效率。

(5)字符串对象的编码

字符串对象的编码可以是**int**、**raw**或者**embstr**。

一.如果一个字符串对象保存的是整数值，且这个整数值可以用**long**类型表示(**8个字节的长整型**)，则**encoding为int**

二.如果保存的是字符串值，且其长度**大于44字节**，则**encoding为raw**

三.如果保存的是字符串值，且长度**小于等于44字节**，则**encoding为embstr**

embstr编码将创建字符串对象所需的内存分配次数，从raw编码的两次降为1次。释放embstr编码的字符串只需调一次内存释放函数，而释放raw编码的要2次。embstr编码的字符串对象所有数据都保存

在一块连续的内存里，能更好利用缓存。

embstr编码的字符串是只读的，如果对embstr编码的字符串进行修改，那么总会变成一个raw编码的字符串对象。

(6)列表对象的编码

列表对象的编码是ziplist或者linkedlist(压缩列表和双端列表)。

列表对象在以下两个条件时，会使用压缩列表ziplist进行编码：

- 一.列表保存的所有字符串元素长度都小于64字节(list-max-ziplist-entries)
- 二.列表保存的元素数量小于512个(list-max-ziplist-value)

当列表对象包含的元素较少时，Redis使用压缩列表作为列表对象的encoding。因为压缩列表比双端链表更节约内存，且在元素较少时，在内存中以连续块方式保存的压缩列表，比起双端链表可以更快地被载入到缓存中。随着列表元素越多，压缩列表的优势逐渐消失，转而使用双端链表。

(7)哈希对象的编码

哈希对象的编码是ziplist或者hashtable(压缩列表和字典)。

压缩列表编码的哈希对象新增键值对时：先将保存了键的压缩列表结点推入到压缩列表表尾，再将保存值的压缩列表结点推入到压缩列表表尾。

哈希对象在以下两个条件时，会使用压缩列表ziplist进行编码：

- 一.哈希保存的所有键值对的键和值的字符串长度都小于64字节(hash-max-ziplist-value)
- 二.哈希保存的键值对数量小于512个(hash-max-ziplist-entries)

(8)集合对象的编码

集合对象的编码可以是inset或者hashtable(整数集合和字典)。

字典编码的集合对象，字典的每个键都是一个字符串对象，每个字符串对象都包含了一个集合元素，而字典的值则全部被设置为NULL。

集合对象在以下两个条件时，会使用inset整数集合进行编码：

- 一.集合保存的所有元素都是整数值
- 二.集合保存的元素数量不超过512个(set-max-intset-entries)

(9)有序集合的编码

有序集合的编码可以是ziplist或者skiplist(压缩列表和跳跃表)。

压缩列表编码的有序集合对象，每个集合元素使用两个紧挨在一起的压缩列表节点来保存。第一个节点保存元素的成员，第二个节点保存元素的分值。

跳跃表编码的有序集合对象，同时使用了跳跃表和字典作为底层实现：

- 一.通过跳跃表，程序可以对有序集合进行范围型操作，如zrank、zrange
- 二.通过字典，程序可以用O(1)复杂度查找给定成员分值，如zscore

虽然同时使用了跳跃表和字典来保存有序集合元素，但这两种数据结构都会通过指针来共享相同元素的成员和分值，所以不会因此而浪费额外的内存。

有序集合对象在以下两个条件时，会使用ziplist压缩列表进行编码：

- 一.有序集合保存的元素小于128个
- 二.有序集合保存的所有元素长度都小于64字节

9.Redis对象的几个关键属性

(1)type属性实现Redis命令的类型检查与多态

(2)refcount属性实现引用计数技术与值共享

(3)lru属性实现空转时长与优先回收

(1)type属性实现Redis命令的类型检查与多态

在执行一个类型特定的命令之前，Redis会通过检查key的值对象redisObject结构的type属性来决定是否执行给定命令。

除了根据值对象的类型来判断是否能执行指定命令外，还会根据值对象的编码方式选择正确的命令实现代码来执行命令。

(2)refcount属性实现引用计数技术与值共享

首先，Redis会通过引用计数技术来实现内存回收机制，而每个对象的引用计数信息，则是由redisObject结构的refcount属性记录的：

- 一.当创建一个新对象时，refcount = 1
- 二.当对象被一个新程序引用时，refcount + 1
- 三.当对象不再被一个程序使用时，refcount - 1
- 四.当refcount = 0时，对象所占的内存会被释放

对象的引用计数refcount，除了可实现引用计数的内存回收机制，还可实现对象共享，节约更多内存。

在Redis中，让多个键共享同一个值对象需要执行以下两个步骤：

- 一.将数据库建的键指针指向一个现有的值对象
- 二.将被共享的值对象的refcount + 1

Redis初始化服务器时，会创建共享值为0到9999的字符串对象。

(3)lru属性实现空转时长与优先回收

除了type、encoding、ptr和refcount四个属性外，redisObject结构还有一个lru属性。lru属性记录了对象最后一次被命令程序访问的时间，查看键的空转时长命令是：redis > object idletime key。

当服务器打开了maxmemory选项，且服务器用于回收内存的算法为volatile-lru或allkeys-lru，则当服务器占用的内存数据超了maxmemory，空转时长较高的那部分键会优先释放回收内存。

10.Redis的单线程为什么这么快

(1)Redis处理命令的过程

(2)为什么单线程这么快

(1)Redis处理命令的过程

Redis是单线程来处理命令的，所以一条命令从客户端到达服务端不会立刻被执行，所有命令都会进入一个队列中，然后被逐个执行。

(2)为什么单线程这么快

一.纯内存访问

Redis将所有数据都放在内存中，内存的响应时长约为100纳秒，这是Redis能达到每秒万级别访问的重要基础。

二.非阻塞IO

使用epoll作为IO多路复用技术的实现，再加上Redis自身的事件处理模型将epoll中的连接、读写、关闭都转换为事件，不在网络IO上浪费过多的时间。

三.单线程

单线程避免了线程切换和竞态产生的消耗。对于服务端开发来说，锁和线程切换通常是性能杀手。单线程对每个命令执行时间是有要求的，如果某个命令执行时间过长，会造成其他命令阻塞。Redis是面向快速执行场景的数据库。

11.Redis的典型应用场景和说明

(1)Redis的字符串的典型应用场景

(2)Redis的列表的典型应用场景

(3)Redis的集合的典型应用场景

(4)Redis的有序集合的典型应用场景

(1)Redis的字符串的典型应用场景

一.缓存功能

二.计数

三.共享Session

四.限速

Redis的字符串值最大不能超过512MB，setnx可以作为分布式锁的一种实现方案。

Redis的批量操作命令如mget，有助于提高效率。但要注意每次批量操作所发送的命令数不是无节制的，如果数量过多可能会造成Redis阻塞或网络拥塞，批量操作也必须是面向快速执行的场景。

incr命令用于计数，很多系统和语言使用CAS机制实现计数功能。使用CAS机制会有一定的CPU开销，但在Redis中不存在该问题。因为Redis是单线程架构，任何命令到Redis服务端都要顺序执行。

(2)Redis的列表的典型应用场景

一.消息队列(lpush + brpop实现阻塞队列)

生产者客户端使用lpush从列表左侧插入元素，多个消费者客户端使用brpop阻塞式的抢到表尾部元素，多个客户端保证了消费的负载均衡和高可用性。

二.文章列表(lrange分页获 + 取排序统计)

lpush + lpop => 栈

lpush + rpop => 队列

lpush + ltrim => 有限集合

lpush + brpop => 消息队列

blpop和brpop是lpop和rpop的阻塞版本。如果列表不为空，客户端会立即返回。如果列表为空，客户端在超时时间内一直等待进行阻塞。

(3)Redis的集合的典型应用场景

sadd => 标签

spop + srandmember => 生成随机数，比如抽奖

sadd + sinter => 社交需求

smembers和lrange、hgetall都属于比较重的命令。如果元素过多，则存在阻塞Redis的可能，这时候可以使用sscan来完成。

(4)Redis的有序集合的典型应用场景

排行榜系统：按时间、查看量、点赞数、进行排行。zadd的时间复杂度为 $O(\log N)$ ，sadd的时间复杂度为 $O(1)$ 。

12.Redis的相关命令说明

(1)Redis的过期命令需要注意

(2)Redis的遍历键命令需要注意

(3)Redis的慢查询命令需要注意

(4)pipeline批量操作命令

(5)watch命令

(6)lua脚本

(7)bitmaps命令

(8)HyperLogLog命令

(9)发布订阅和GEO

(10)时间序列数据的处理

(1)Redis的过期命令需要注意

一.如果expire key的键不存在，则返回结果为0

二.如果过期时间为负值，那么键会立即被删除，和del命令一样

三.persist命令可将键的过期时间清除

四.对于字符串类型键，执行set命令会去掉过期时间

五.Redis不支持二级数据结构(比如哈希、列表)内部元素的过期

六.setex == set + expire，该setex原子操作

(2)Redis的遍历键命令需要注意

一般不要在生产环境下使用keys命令，如果实在需要遍历键，那么可以：

一.在一个不对外提供服务的Redis从节点上执行，这样就不会阻塞客户端，但影响主从复制

二.如果确认键的总数确实比较少，则可执行

三.使用scan渐进式便利替代，可以有效防止阻塞

如果scan过程中有键的变化(增删改)，那么scan并不能保证遍历出所有键。

哈希遍历：hgetall => hscan

集合遍历：smembers => sscan

(3)Redis的慢查询命令需要注意

一.慢查询中的两个重要参数：slowlog-log-slower-than和slowlog-max-len

二.慢查询不包含命令网络传输和排队时间

三.慢查询日志是一个先进先出的队列，线上可调大队列长度为1000，慢查询时间为1毫秒

四.可定期执行slow get命令将慢查询日志持久化到MySQL

(4)redis-cli的重要选项

一.--bigkeys：使用scan命令对Redis键采样，从中找出内存占用较大的键值

二.--latency：检测网络延迟

redis-benchmark可以为Redis做基准性能测试，sysbench是数据库压测。

(4)pipeline批量操作命令

pipeline可以有效减少RTT(往返时间)次数，但每次pipeline的命令数量不能没节制。原生批量命令是原子的，pipeline是非原子的。原先批量命令是一个命令对应多个key，pipeline支持多个命令。

(5)watch命令

Redis不支持事务中的回滚功能。Redis提供watch命令，在事务中使用(multi)来实现乐观锁。

(6)lua脚本

Redis可以使用lua脚本创造出原子、高效、自定义的命令组合。Redis执行lua脚本的方法是：eval和evalsha。

(7)bitmaps命令

bitmaps可以用来做独立用户统计，有效节省内存。bitmaps中setbit一个大的偏移量，由于申请大量内存会导致阻塞。

(8)HyperLogLog命令

HyperLogLog实际类型为字符串类型，它是一种基数算法。通过HyperLogLog可以利用极小的内存空间完成独立总数的统计，但存在0.81%的失误率。

(9)发布订阅和GEO

Redis的发布订阅无法实现消息堆积和回溯，也不会对发布的消息进行持久化。Redis的GEO功能，底层的实现是zset，可用来实现基于地理位置时信息的应用。

(10)时间序列数据的处理

特点：快速写入，能进行根据时间查询、根据时间范围查询、根据时间范围聚合计算。

方案一：组合使用Hash和Sorted Set，把数据同时保存在Hash集合和Sorted Set集合。通过Hash保证按时间查询，通过Sorted Set保证按时间范围查询。如果要进行聚合计算，则需要将数据传输到客户端进行聚合计算。

方案二：使用Redis的扩展模块，专门为存取时间序列数据设计的RedisTimeSeries模块。

Redis 17 缓存 8

[Redis · 目录](#)

[下一篇 · Redis原理—2.单机数据库的实现](#)

Reads 356
